The University of Manchester

Department of Computer Science

Project Report

Milestone 1

Modernising the NVDLA Software Stack for Linux 6.6

Author: Berkant Bakisli

Date: 10 July 2026

Modern Linux KMD/UMD implementation and virtual-platform correctness validation

Abstract

The first project milestone was to make the open-source NVIDIA Deep Learning Accelerator (NVDLA) kernel-mode driver (KMD) and user-mode runtime (UMD) work correctly on a modern ARM64 Linux system. The upstream software was written for a Linux 4.13-era environment and could not be built or executed unchanged against Linux 6.6. A successful compilation alone would not establish correctness because the driver is responsible for the DRM userspace interface, GEM buffer allocation, DMA address translation, task submission, scheduling, and interrupt handling.

This milestone therefore combined an upstreamable software forward port with a reproducible virtual-platform test environment. The work produced a nine-patch KMD/UMD series against pinned upstream NVDLA software, a Buildroot ARM64 toolchain and root filesystem, a Linux 6.6 kernel, target-side smoke tests, and deterministic runtime workloads. The final `nv_small` validation used a source-built VP and CMOD, a KMD built for the small register map, and a LeNet loadable compiled for `nv_small`. Ten consecutive inference runs completed 100 hardware-layer operations and returned the expected vector `0 2 0 0 0 0 0 124 0 0` in every run. No kernel or VP fault pattern was detected. These results establish the software stack as a suitable basis for the next integration milestone, while leaving physical DMA, interrupt routing, and reset behaviour for later board-level validation.

Contents

1 Introduction

1.1 Motivation

The NVDLA software stack provides the path between a compiled neural-network loadable and the accelerator hardware. The UMD parses the loadable and manages runtime execution, while the KMD exposes a DRM render node, allocates GEM buffers, translates submitted memory handles, schedules accelerator tasks, and services completion interrupts. Consequently, a hardware implementation is of limited practical use unless both software layers operate correctly on the target operating system.

The upstream NVDLA software revision used in this project predates the Linux 6.x DRM and DMA-BUF interfaces. Direct compilation exposed removed headers, changed function signatures, renamed GEM helpers, obsolete callbacks, and unavailable coherent-memory functions. Even after these compile failures were resolved, execution correctness still depended on details that a compiler could not check, including interrupt discovery, GEM `mmap` dispatch, the NVDLA hardware configuration, and whether DMA buffers were allocated in a memory region visible to the virtual accelerator.

The milestone was therefore framed as a correctness problem rather than only a porting problem. The intended outcome was a modern stack that could build, boot, load, execute a real CNN, and return the same result as a controlled stock environment.

1.2 Aim and Objectives

The aim of this milestone was to modernise the NVDLA KMD and UMD for Linux 6.6 without changing the established userspace ABI, and to demonstrate functional correctness in an NVDLA virtual platform.

The objectives were:

1. Create a reproducible ARM64 Linux 6.6 build environment.
2. Preserve a pristine upstream `nvdla/sw` checkout and maintain changes as an upstream-style patch series.
3. Forward-port the KMD to current kernel, DRM, GEM, DMA-BUF, IRQ, and reserved memory interfaces.
4. Build the UMD runtime and runtime library with the same pinned ARM64 toolchain.
5. Preserve the NVDLA DRM ioctl numbers, structures, and KMD/UMD contract.
6. Validate module probe and GEM memory operations before submitting work to the accelerator.
7. Execute a deterministic CNN through the complete compiler, UMD, KMD, scheduler, interrupt, and CMOD path.
8. Verify that the VP, KMD, and model loadable all target `nv_small`.
9. Archive enough metadata and logs to reproduce and audit each result.

1.3 Success Criteria

Table 1 defines the criteria used to decide whether the milestone had been completed. These criteria deliberately extend beyond successful compilation.

Criterion	Required evidence
Reproducible build	Pinned source revisions, recorded toolchain identity, and successful kernel, rootfs, KMD, and UMD build manifests.
ABI preservation	Matching KMD and UMD ioctl definitions and structure names with no public ioctl renumbering.

Criterion	Required evidence
Driver probe	`opendla.ko` loads, the expected NVDLA compatible string is reported, and `/dev/dri/renderD128` is created.
GEM correctness	GEM create, map-offset, `mmap`, CPU read/write, and destroy complete without an Oops or warning.
Runtime correctness	A real CNN completes all hardware layers and produces the expected output exactly.
Configuration correctness	VP/CMOD, KMD register map, and compiler target all identify `nv_small`.
Repeatability	Repeated runs return identical output and contain no kernel or VP fault patterns.
Evidence quality	Inputs, binaries, outputs, logs, hashes, and pass/fail reasons are recorded in per-run manifests.

1.4 Scope

This chapter covers the modern Linux software stack and its VP validation. It does not claim that the VP reproduces every property of the physical FPGA system. In particular, the VP cannot establish the behaviour of the real non-coherent memory path, physical interrupt wiring, FPGA reset sequencing, or hardware timing. Those are separate acceptance tests for the next project stage.

The milestone boundary is the nine-patch series from `0001` to `0009` under `patches/nvdla-sw/`. Patches introduced during the following integration stage are intentionally excluded from this account.

2 Background

2.1 NVDLA Software Execution Path

The software execution path used in this work is illustrated in Figure 1. A model is first converted to an NVDLA loadable by the compiler. On the target, `nvdla_runtime` and `libnvdla_runtime.so` parse this loadable and communicate with the KMD through a DRM render node. The KMD allocates GEM objects, exports memory references, translates submitted handles into accelerator-visible DMA addresses, and invokes the common NVDLA firmware scheduler. Completion is reported through the NVDLA interrupt and returned to the UMD.

```
 Caffe model and calibration data
              |
              v
        NVDLA compiler
              |
              v
      nv_small loadable + input image
              |
              v
  nvdla_runtime / libnvdla_runtime.so       User space
              |
         DRM ioctls and GEM buffers
              |
              v
           opendla.ko                       Kernel space
              |
      CSB programming, DMA and interrupt
              |
              v
        NVDLA VP / nv_small CMOD
```

Figure 1: End-to-end software path exercised by the correctness gate.

This path is important to the validity of the final test. A correct output from LeNet requires substantially more than register access: the loadable must be parsed, multiple buffers must be allocated and populated, the KMD must submit the correct addresses, the firmware must schedule each hardware layer, and the interrupt path must complete each task.

2.2 Compatibility Gap

The pinned upstream `nvdla/sw` base is commit `79538ba1b52b040a4a4645f630e457fa01839e90`. Its Linux port assumes interfaces available in the original VP environment. The first Linux 6.6 builds exposed the following classes of incompatibility:

- inclusion of userspace `stdarg.h` during a kernel build;
- the changed `dma_buf_vmap()` and `dma_buf_vunmap()` interface using `iosys_map`;
- replacement of CMA-specific DRM GEM helpers with DMA GEM helpers;
- removed GEM reference-counting and driver callbacks;
- read-only virtual-memory flags requiring helper accessors;
- unavailable legacy coherent-memory declaration functions;
- DMA-BUF namespace requirements;
- changed platform IRQ discovery behaviour; and
- a different modern GEM `mmap` dispatch path.

The UMD also exposed modern C++ and linker assumptions. It used `std::numeric_limits` without directly including `<limits>`, and the runtime executable linked a non-PIC static JPEG library with a toolchain that defaults to position-independent executables.

2.3 Why Compilation Was Not Sufficient

Several important faults appeared only after the module had built. Legacy IRQ resource lookup did not obtain the translated device-tree interrupt on the modern kernel. An initial GEM compatibility implementation also routed the GEM object callback back through a high-level `drm_gem_mmap()` helper, creating recursive dispatch and a kernel Oops when userspace mapped a buffer.

Later, the runtime completed all LeNet hardware layers and reported a passing test while returning `12 12 12 12 12 12 12 12 12 12` rather than the stock output. This was a particularly useful counterexample: clean module loading, successful scheduling, and a zero exit status did not imply tensor correctness. Exact output comparison was therefore necessary.

3 Methodology

3.1 Reproducible Build Environment

The build and test environment was designed around pinned inputs and separate generated directories. Large sources and build products were stored on the WSL ext4 filesystem to avoid Windows path restrictions and reduce kernel build times. Source metadata was recorded in `repro.lock.json`, while generated sources and work products were kept outside Git.

Table 2 summarises the environment represented by the successful VP evidence.

Component	Tested version or revision
Architecture	ARM64 (`aarch64`)
Kernel	`linux-xlnx` 6.6.80+, commit `e29e392a451244a11aa3559738b6617536fde460`
Build system	Buildroot 2024.02.11, tested checkout `4d772ebe75a922695e3c0c71ae7236c89ba76e01`
Compiler	Buildroot GCC and G++ 12.4.0
NVDLA software	commit `79538ba1b52b040a4a4645f630e457fa01839e90` plus patches `0001-0009`
NVDLA VP	commit `f7ce663b95adf4f381de186b665becae28df26ed`
NVDLA hardware/CMOD	`nv_small` commit `771f20cc9e69759d7277978eb41e8d47f1547374`
VP support environment	Locked `nvdla/vp:latest` Docker image, used for SystemC and host libraries

Buildroot generated both the C and C++ cross compilers. The same toolchain was used for the modern kernel support utilities, KMD, UMD runtime, and target-side smoke program. The modern root filesystem included an autorun hook so automated tests could mount a host payload, load the module, run the selected test, collect logs, and shut down without depending on serial-login timing.

The legacy VP loader could not boot the minimal raw ARM64 kernel image because its header used a zero text offset that overlapped the loader region. The build therefore retained the raw `Image` and generated `Image.vp2m` with a 2 MiB text offset for VP execution. This changed only the ARM64 image header required by the loader.

3.2 Upstreamable Patch Workflow

The upstream source checkout under `.external/sources/nvdla-sw` was kept pristine. A separate `.work/nvdla-sw-patched` Git worktree received one logical change per commit. The commits were exported with `git format-patch` and stored under `patches/nvdla-sw/`.

This separation prevented VP- or board-specific values from entering the common driver. In particular, the KMD did not gain hard-coded physical addresses, forced coherent-DMA declarations, Docker paths, or reset policy. Platform memory remained a firmware and device-tree responsibility. Each patch was checked for application against the pinned base, subjected to kernel `checkpatch` where available, built with warnings visible, and accompanied by a problem/cause/fix style commit message.

3.3 KMD Forward Port

The KMD was advanced in small steps so that each build failure exposed the next obsolete interface. Table 3 summarises the milestone patches.

Patch	Change	Reason
0002	Use linux/stdarg.h	Kernel builds no longer exposed the userspace header expected by the callback code.
0003	Add legacy pointer and modern iosys_map DMA-BUF copy helpers	Linux 6.6 changed the vmap API while the task ABI remained unchanged.
0004	Move to DMA GEM helpers and modern GEM object callbacks, reference helpers, and VM flag accessors	CMA helper names and several driver-level callbacks were removed or relocated.
0005	Use platform_get_irq() and correct the modern GEM mmap path	Device-tree IRQ translation failed through the legacy lookup, and recursive mmap dispatch caused an Oops.
0008	Add NVDLA_HW_CONFIG=initial|small	External module builds otherwise selected the initial register map implicitly.
0009	Attach optional device-tree reserved memory on Linux 6.x	The modern path no longer used the legacy declared coherent-memory mechanism.

Compatibility conditionals were kept close to the affected API. Older-kernel paths were retained where the maintenance cost remained small. The public NVDLA ioctl structures and numbers were not altered.

3.4 UMD and Runtime Forward Port

The userspace patches were intentionally narrow:

- 0001 retained /dev/dri/renderD128 as the default but allowed NVDLA_DEVICE_NODE to override it. This made tests deterministic on systems with more than one DRM device without changing default behaviour.
- 0006 included <limits> at the point where std::numeric_limits is used, allowing the source to build reliably with GCC 12.4.0.
- 0007 introduced an optional RUNTIME_LDFLAGS variable. The Buildroot lane passed -no-pie when linking the runtime executable against the upstream non-PIC static JPEG library; the shared NVDLA runtime library was unchanged.

The successful build produced both nvdla_runtime and libnvdla_runtime.so. readelf summaries and binary hashes were included in the build manifests so that the target architecture and dynamic dependencies could be audited.

3.5 Staged Correctness Tests

Testing progressed from narrow, deterministic checks to full inference. This reduced the number of possible causes at each failure. Table 4 summarises the resulting test ladder.

Stage	Test	Faults isolated
1	Patch application and ABI audit	Patch drift, ioctl renumbering, or KMD/UMD structure mismatch.
2	Linux 6.6 KMD build	Removed or changed kernel APIs and module symbol problems.
3	VP boot and module probe	Kernel image, rootfs, device tree, compatible string, IRQ, and render-node creation.
4	GEM smoke utility	GEM create, map-offset, mmap, CPU read/write, and destroy.
5	Runtime server and flatbuffer client	UMD/KMD protocol, task submission, scheduler, and interrupt completion.
6	LeNet/MNIST exact comparison	End-to-end model execution and tensor correctness.
7	Repeated LeNet execution	Determinism and state retained across multiple runs in one boot.

Every VP run scanned the serial and kernel logs for `Oops`, `BUG`, `WARNING`, `DMA-API`, scheduler timeout, interrupt timeout, unexpected reset, CMOD fatal messages, and `TLM_ADDRESS_ERROR_RESPONSE`. A test could only pass when its functional result and log scan both passed.

3.6 Stock Controls and Hardware-Configuration Matching

NVDLA configuration is selected independently in three places: the VP/CMOD, the KMD register header, and the compiler target used for the loadable. Early tests showed that treating the stock Docker VP as `nv_small` produced misleading failures. With a small-target loadable and the available stock modules, the VP either terminated during convolution or reported invalid CSC/CDMA programming and stalled. In contrast, a loadable compiled for `nv_full` completed on the stock `nvdla/vp:latest` environment and produced the expected digit-7 vector.

The stock image was therefore classified as an `initial/nv_full` control, not an `nv_small` oracle. The test harness recorded workload compatibility and rejected a result when the loadable target did not match the probed KMD configuration.

For the final lane, the NVDLA VP and hardware repositories were pinned and the CMOD was built explicitly with `NVDLA_HW_PROJECT=nv_small`. The resulting `aarch64_toplevel` and `libnvdla_cmod.so` were mounted into the locked Docker environment, which supplied the compatible SystemC installation. This retained the reproducibility of a source-built small hardware model without treating a Docker runtime setting as a hardware-configuration switch.

3.7 Debugging Runtime Data Correctness

The first modern `nv_full` LeNet run was a valuable intermediate failure. The KMD loaded, `/dev/dri/renderD128` existed, all ten hardware layers completed, and the runtime returned success. Nevertheless, the output was the constant vector `12 12 12 12 12 12 12 12 12 12` rather than the stock result.

An opt-in local tracing patch was used for diagnosis but was deliberately kept out of the upstream patch series. It showed that CPU-visible GEM buffers contained the expected loadable and image data, while the programmed DMA addresses were in ordinary QEMU RAM near `0x6ec00000`. Changing write-combine attributes, forcing a coherent flag, and adding explicit DMA synchronisation did not alter the output, so these experiments were discarded.

A negative control moved the VP RAM target to the extmem range while leaving the GEM allocations in ordinary QEMU RAM. NVDLA then reported `TLM_ADDRESS_ERROR_RESPONSE` when reading the `0x6ed...` addresses. This linked the incorrect output to the VP memory topology rather than to the loadable or arithmetic path.

The first device-tree reserved-memory experiment allocated a 128 MiB `no-map` `shared-dma-pool` at `0x70000000` in ordinary QEMU RAM. Patch `0009` attached the pool successfully and GEM addresses moved into it, but the output remained all 12s. The decisive test moved the same pool into the VP external-memory aperture at `0xc0000000..0xc7ffffff` and configured the VP RAM target as `0xc0000000..0xffffffff`. The modern KMD then allocated buffers visible to both the CPU and CMOD, and the LeNet output matched the stock result exactly.

This result justified the final design: the common KMD only attaches an optional firmware-described memory region, while the VP-specific address and size remain in the VP device tree.

3.8 Deterministic LeNet Workload

The final workload used the Columbia LeNet/MNIST model, calibration table, and `seven.pgm` input. All source files were pinned by SHA-256. The loadable was compiled with the locked stock compiler using:

```
--profile fast-math
--cprecision int8
--configtarget nv_small
--quantizationMode per-filter
--informat nchw
```

The generated loadable was 445,736 bytes with SHA-256 `4b6aa87846329e46e9468a37ffdc0a111884fa63390cdfbea26a95a455edb29d`. The expected ten-element output was fixed before the modern run. A pass required all ten hardware layers to complete, exact equality with this vector, and an empty bad-pattern scan.

4 Results and Evaluation

4.1 Build and ABI Results

The pinned Buildroot toolchain successfully produced a Linux 6.6.80+ ARM64 kernel, root filesystem, `opendla.ko`, `nvdla_runtime`, and `libnvdla_runtime.so`. The small-config module used for the final lane had vermagic `6.6.80+ SMP aarch64` and SHA-256 `d323d6f3482bd3bbaf0efd823548be72458cc911d131b3593d74af2d9dc1e4a0`.

The ABI audit passed. KMD and UMD contained the same four public DRM ioctl definitions for GEM create, GEM destroy, GEM mmap offset, and task submit, and the same six relevant structure names. The two header files were not byte-identical, so the audit emitted a warning, but their extracted semantic ABI facts matched. No compatibility patch changed an ioctl number or public structure.

4.2 Modern KMD Smoke Result

The final `nv_small` smoke run booted the source-built VP with the modern kernel and root filesystem. The KMD reported `nvidia,nv_small`, attached the reserved memory pool at `0xc0000000`, and created `/dev/dri/renderD128`. The target-side utility then completed GEM create, map-offset, `mmap`, CPU read/write verification, and destroy. The run contained no classified kernel or VP fault pattern.

This test directly confirmed the two runtime-sensitive KMD fixes introduced after compilation: translated IRQ lookup and non-recursive GEM mmap dispatch.

4.3 `nv_full` Control Result

Before promoting `nv_small` as the primary lane, the patched modern stack was compared with the working stock `nv_full` path. With the extmem-backed DMA pool, the clean modern module completed all ten LeNet layers and returned:

```
0 2 0 0 0 0 0 124 0 0
```

This exactly matched the stock output and replaced the earlier all-12 result. The control demonstrated that the forward-ported KMD and UMD could preserve known runtime behaviour when the hardware configuration and VP memory topology were held consistent.

4.4 Primary `nv_small` Correctness Result

The formal `nv_small` gate validated each independently selected configuration layer. Table 5 records the configuration proof.

Layer	Recorded value	Result
VP CMake project	`NVDLA_HW_PROJECT=nv_small`	Pass
Linked CMOD	Source-built `nv_small/libnvdla_cmod.so`	Pass
Device-tree/KMD probe	`nvidia,nv_small`	Pass
KMD build option	`NVDLA_HW_CONFIG=small`	Pass
Loadable target	`nv_small`	Pass
Render node	`/dev/dri/renderD128`	Pass
VP DMA aperture	`0xc0000000..0xffffffff`	Pass

The corresponding audit also verified SHA-256 identities for the VP binary, CMOD, DTB, and module, and confirmed through `ldd` that the source-built `aarch64_toplevel` resolved the intended small CMOD.

The final test executed LeNet ten times in a single boot. All ten runs passed, all 100 hardware-layer operations completed, and no first failing repeat was recorded. Every output was:

```
0 2 0 0 0 0 0 124 0 0
```

Each output file had the same SHA-256, `0f3bf507cf40a21a099bf70663a59a62b75114d3d65c633e80ddb90ab73401e1`. The highest value was at index 7, corresponding to the expected classification of the input image. No Oops, BUG, warning, DMA-API error, timeout, reset, CMOD fatal, or TLM address error was classified in the run.

Table 6 summarises the main result.

Metric	Result
Requested repeats	10
Passed repeats	10
Completed hardware-layer operations	100
Expected output matches	10/10 exact
Distinct output hashes	1
First failing repeat	None
Bad kernel/VP patterns	0
Overall classification	Pass

4.5 SDP Regression Diagnostic

The upstream `sdp_regression_small` flatbuffer was initially intended as a minimal runtime workload. In the final small lane, the KMD loaded, the render node existed, the runtime server accepted the request, SDP programming and completion markers were observed, and a 2,084-byte `.dimg` was returned. However, the payload following its header was entirely zero and did not match the selected upstream golden output.

This did not provide evidence against the modern driver because an equivalent stock `nv_full` control exhibited the same important behaviour: the stock runtime reported a passing test and returned an output, but its payload was also zero and failed exact comparison with the selected upstream golden. The SDP workload was therefore retained as a diagnostic of module loading, protocol, scheduling, and interrupt completion, but it was not used as the primary tensor-correctness oracle.

LeNet was the stronger criterion because it passed in a matching stock control, exercised ten hardware layers, and produced a directly interpretable output that could be compared exactly across stock and modern environments.

4.6 Evaluation Against the Success Criteria

Table 7 evaluates the milestone against the criteria from Section 1.3.

Criterion	Evaluation
Reproducible build	Met. Build manifests record the source SHAs, compiler identity, binary hashes, and logs.
ABI preservation	Met. Semantic ioctl and structure checks pass with no public ABI changes.
Driver probe	Met in VP. The small compatible string and render node were recorded.

Criterion	Evaluation
GEM correctness	Met in VP. All smoke stages completed without a bad kernel pattern.
Runtime correctness	Met in VP. LeNet completed and matched the stock expected output exactly.
Configuration correctness	Met. VP/CMOD, KMD, device tree, and loadable were independently audited as `nv_small`.
Repeatability	Met for ten consecutive runs. The planned 100-repeat extended gate remains available but was not completed in the archived milestone evidence.
Evidence quality	Met. Manifests, serial output, dmesg, module logs, workload hashes, and output hashes were archived.

The evidence supports the conclusion that the modernised stack is functionally correct within the VP model. The conclusion is narrower than claiming general hardware correctness: it establishes the Linux-facing driver and runtime path under a deterministic emulated NVDLA platform.

5 Critical Reflection and Threats to Validity

5.1 Strengths of the Validation

The strongest aspect of the methodology was the use of progressively broader tests. Compilation isolated API changes, the GEM utility exposed a real mmap Oops, configuration controls prevented mismatched loadables from being treated as driver failures, and LeNet exposed a DMA visibility problem despite a clean runtime exit. This progression made failures explainable and prevented a single weak indicator from being treated as proof.

The stock controls were also important. The stock SDP mismatch showed that an upstream regression artifact was not automatically a valid golden test. The stock `nv_full` LeNet result, by contrast, provided a known behaviour that the modern stack could reproduce before the work moved to the source-built `nv_small` lane.

Finally, keeping diagnostic instrumentation outside the upstream patch queue helped preserve a small and maintainable change set. Experiments that did not change the result were documented through artifacts but not retained as common driver behaviour.

5.2 Limitations

The primary limitation is that a CMOD is not the physical FPGA. The VP supports the relevant register, DMA, scheduler, and interrupt interactions, but it does not reproduce every cache, coherency, reset, or timing property of the target system. The extmem requirement is specifically a VP memory-topology result and must not be copied as a fixed physical-address rule for another platform.

The archived stability evidence contains ten consecutive LeNet runs rather than the planned 100-run extended gate. Ten identical runs provide useful evidence that state is cleaned up between executions, but a longer run is still desirable before the final dissertation evaluation.

The SDP regression remains unresolved as a tensor oracle. Its stock-comparable zero output makes it unsuitable for attributing failure to the modern driver, but the relationship between its selected loadable and golden file should be investigated separately.

The ABI audit compares extracted ioctl definitions and structure names rather than performing a full compiler-generated layout comparison for every build architecture. A future extension could compile a shared KMD/UMD probe that records `sizeof` and field offsets directly.

5.3 Reproducibility Note

The successful VP manifests and the checked-out source identify Buildroot 2024.02.11 as commit `4d772ebe75a922695e3c0c71ae7236c89ba76e01`. The current `repro.lock.json` records the same tag but a different commit value. This metadata should be reconciled before a clean-room reproduction is used as final dissertation evidence. The tested revision reported in this chapter is the revision captured by the successful build manifests.

6 Summary of Achievements

This milestone produced a working NVDLA KMD and UMD for the Linux 6.6 VP environment. The KMD was updated for modern kernel, DRM, GEM, DMA-BUF, IRQ, and reserved-memory interfaces while preserving the userspace ABI. The UMD was made buildable with the modern C++ toolchain and configurable enough to select a deterministic DRM render node. These changes were organised as small, upstream-style commits against a pinned NVDLA software base.

A reproducible validation framework was built around the forward port. It generated the ARM64 toolchain, kernel, root filesystem, module, runtime, and test payloads; booted stock and modern VP lanes; collected serial and kernel logs; compared outputs; and archived source and binary identities.

Most importantly, the final result went beyond compilation and module loading. A verified source-built `nv_small` VP, small-config KMD, and small-target LeNet loadable executed ten consecutive inferences correctly. All 100 hardware-layer operations completed, every output matched the expected digit-7 vector, and no classified kernel or VP fault was recorded. This satisfies the first project milestone and provides a controlled software baseline for later integration and physical-hardware validation.

7 Recommended Figures and Listings for the Final Report

The following repository evidence can be converted into figures or listings when this draft is incorporated into the final typeset report:

1. A diagram based on Figure 1 showing compiler, UMD, DRM/GEM, KMD, and CMOD.
2. A shortened listing from patch `0005` showing `platform_get_irq()` and the modern GEM mmap callback selection.
3. A shortened listing from patch `0009` showing optional reserved-memory attachment.
4. A serial-log extract showing `Probe NVDLA config nvidia,nv_small`, reserved memory assignment, and `/dev/dri/renderD128`.
5. A plot or table of all ten LeNet output hashes and completion counts.
6. A comparison diagram of ordinary QEMU RAM and the passing VP extmem DMA aperture.

Evidence Index

The generated `artifacts/` directories are intentionally excluded from Git, but the following run identifiers provide the source evidence for this draft:

Evidence	Run or tracked source	Purpose
E1	`repro.lock.json`	Pinned upstream, Docker, and workload metadata.
E2	`artifacts/abi-check.json`	Semantic KMD/UMD ioctl ABI comparison.
E3	`20260702T154815Z-vp-toolchain` through `20260702T154857Z-vp-runtime`	Passing ARM64 toolchain, kernel, rootfs, KMD, and UMD build evidence.
E4	`20260702T154938Z-vp-modern-smoke`	First complete modern GEM smoke pass.
E5	`20260707T204212Z-vp-modern-lenet-full`	Passing modern `nv_full` LeNet control with extmem-backed DMA.
E6	`20260708T144613Z-vp-modern-smoke`	Passing source-built `nv_small` probe and GEM smoke.
E7	`20260708T194300Z-vp-modern-lenet-small`	Ten-repeat primary `nv_small` LeNet correctness result.
E8	`20260708T194754Z-vp-small-config-audit`	VP, CMOD, DTB, KMD, and probe configuration proof.
E9	`20260709T192642Z-vp-modern-runtime`	Classified small SDP zero-output diagnostic.
E10	`20260709T191532Z-vp-stock-sdp-full`	Stock SDP control showing the same golden-output limitation.
E11	`patches/nvdla-sw/0001` through `0009`	Upstreamable software changes included in this milestone.

References

[1] NVIDIA, *NVDLA Software*, Git repository, base commit `79538ba1b52b040a4a4645f630e457fa01839e90`.

[2] AMD/Xilinx, *linux-xlnx*, commit `e29e392a451244a11aa3559738b6617536fde460`.

[3] NVDLA, *Virtual Platform*, commit `f7ce663b95adf4f381de186b665becae28df26ed`.

[4] NVDLA, *Hardware Repository, nv_small configuration*, commit `771f20cc9e69759d7277978eb41e8d47f1547374`.

[5] J. U. Georgis, *Evaluating Deep Learning Acceleration on FPGA: NVDLA Case Study*, University of Manchester Project Report, 2025.